



Lecture 2: Data Preparation

Cleaning, Scaling, Encoding

Omkar Bait (CosmicAI Fellow)
NRAO, Charlottesville

Garbage In, Garbage Out

No model — however clever — can rescue badly prepared data.

1. Clean

- Missing values → impute or drop
- Outliers → detect & decide
- Duplicates → remove

2. Transform

- Scale: standardize / min-max
- Encode: nominal vs ordinal
- Log-transform skewed features

3. Reduce

- Drop redundant / leaky features
- PCA (optional)
- Result: a clean X matrix

Punchline: data prep is usually **60–80%** of the work in any real ML project. It's also where bugs hide.

Missing Values — What & Why

Real datasets are full of NaN, blank cells, sentinel values like -99.

Why values go missing (astro examples):

Sensor saturation — too bright for the detector

Below detection limit — too faint to measure

Bad pixels / cosmic rays — masked in pipeline

Human error — a column was never populated

Flavors of missing data

Missing Completely At Random:
unrelated to anything in the data.

Missing At Random:
depends on other observed features.

Systematically Missing:
*depends on the missing value itself —
e.g. faint galaxies missing because faint.*

First step: always run `df.isna().sum()`. Decide column-by-column whether to drop, impute, or flag.

Imputation — Mean, Median, Mode

Simplest fix: replace the missing value with a typical value from the column.

Mean

when to use

Numeric, roughly symmetric

example

filling missing redshift z
with the dataset mean $\bar{z}$

Median

when to use

Numeric, skewed or with
outliers

example

filling missing magnitudes
with the column median

Mode

when to use

Categorical features

example

filling missing galaxy type
with the most common class

```
from sklearn.impute import SimpleImputer  
imp = SimpleImputer(strategy="median"); X = imp.fit_transform(X)
```

Worked Example — Pick the Imputer

WORKED
EXAMPLE

Scenario

You have **7 valid r-band magnitudes** in a small sub-sample:

[18.1, 18.0, 17.9, 18.2, 18.1, 18.0, 25.0]

The 25.0 is a suspected outlier (likely near the detection limit). One additional galaxy is missing its r-band measurement.

Compute on paper:

1. The mean of the 7 values.
2. The median of the 7 values.
3. Which would you use to impute the missing galaxy, and why?

Pause and try — then advance.

Worked Example — Solution

ANSWER

Step 1 — Mean

```
sum  = 18.1 + 18.0 + 17.9 + 18.2  
      + 18.1 + 18.0 + 25.0  
      = 133.3  
mean = 133.3 / 7 = 19.04 mag
```

Step 2 — Median

```
sort:  
  [17.9, 18.0, 18.0,  
   18.1, 18.1, 18.2, 25.0]  
median = 4th value = 18.1 mag
```

Step 3 — Decision

The outlier 25.0 pulled the mean up to **19.04** — far from any typical galaxy in the sample. The median **18.1** is robust to the outlier. → **use the median.**

Rule of thumb: median if you suspect outliers or a skewed distribution. Mean only when the data is clean without outliers.

Imputation — KNN

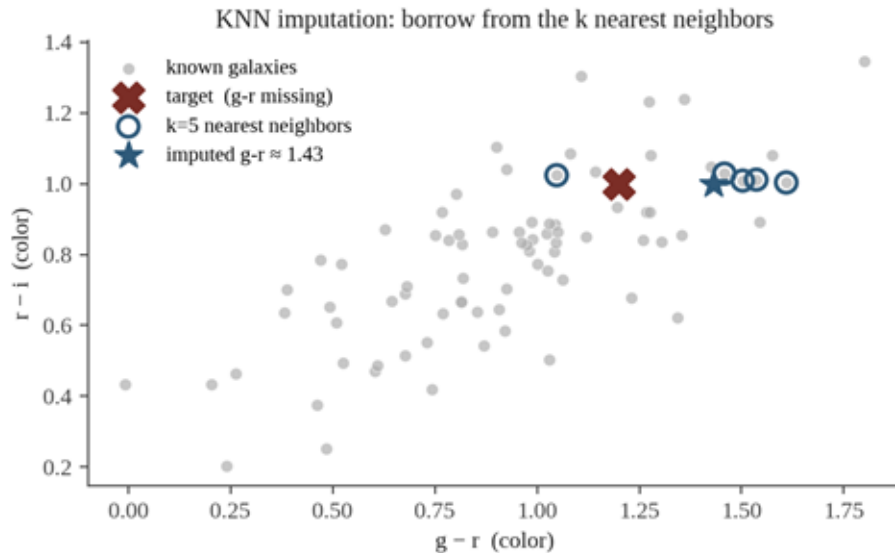
Idea: use the k most similar examples (in the other features) to estimate the missing one.

Steps:

1. Define a distance on the non-missing features.
2. Find the k nearest neighbors.
3. Impute = mean (or median) of their values.

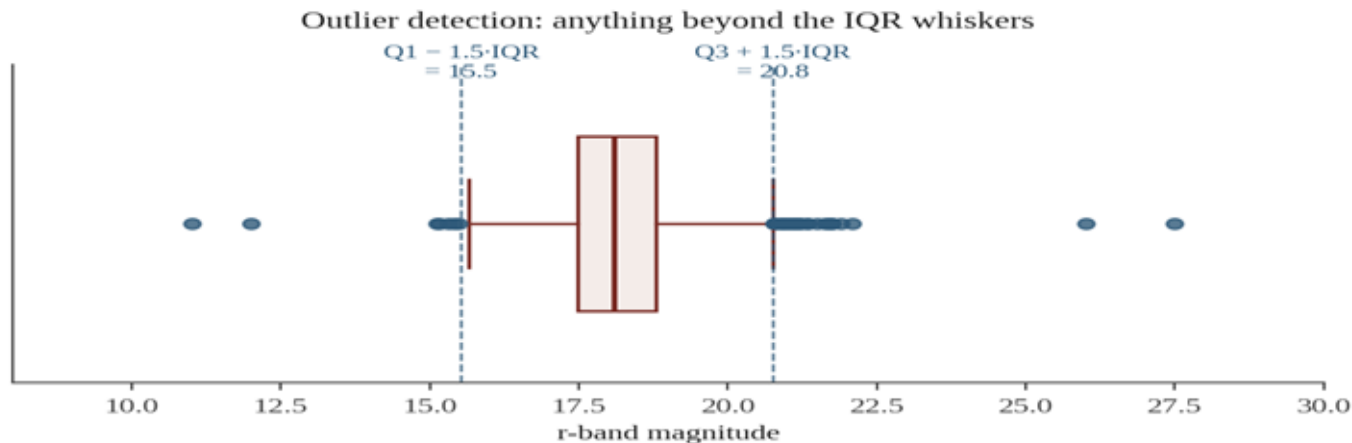
Pros: uses structure in your data, not just one global statistic.

Cons: slow for big datasets; sensitive to feature scaling (do it first!).



```
from sklearn.impute import KNNImputer
X = KNNImputer(n_neighbors=5).fit_transform(X)
```

Outlier Detection — Z-score & IQR



Z-score method

$$z = (x - \mu) / \sigma$$

Flag if $|z| > 3$ ($\approx 0.3\%$ of a normal).

Assumes roughly Gaussian.

IQR method (more robust)

$$\text{IQR} = Q3 - Q1$$

Flag $x < Q1 - 1.5 \cdot \text{IQR}$ or $x > Q3 + 1.5 \cdot \text{IQR}$.

No Gaussian assumption.

Flagged $\neq$ bad. Always investigate before deleting — the outlier might be a quasar, a lens, or a transient.

Worked Example — Z-score Outlier

WORKED
EXAMPLE

Scenario

From a **clean calibration sample** of ~1000 galaxies you have established:

$$\mu_r = 18.0 \text{ mag} \qquad \sigma_r = 0.5 \text{ mag}$$

You measure a **new galaxy** with $r = 19.8 \text{ mag}$.

Compute on paper:

1. The Z-score for the new galaxy.
2. Is it an outlier under the $|Z| > 3$ rule?

Reminder: $Z = (x - \mu) / \sigma$

Pause and try — then advance.

Worked Example — Solution

ANSWER

Step 1 — Apply the formula

$$Z = (x - \mu) / \sigma$$

$$Z = (19.8 - 18.0) / 0.5$$

$$Z = 1.8 / 0.5 = \mathbf{3.6}$$

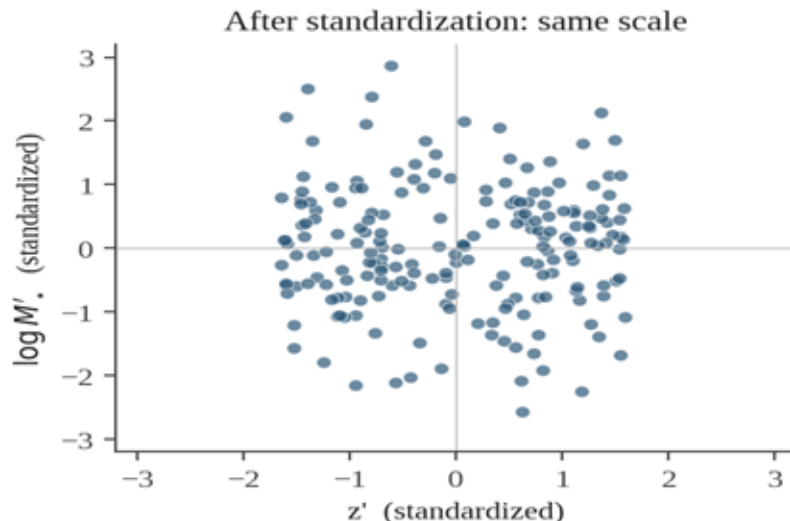
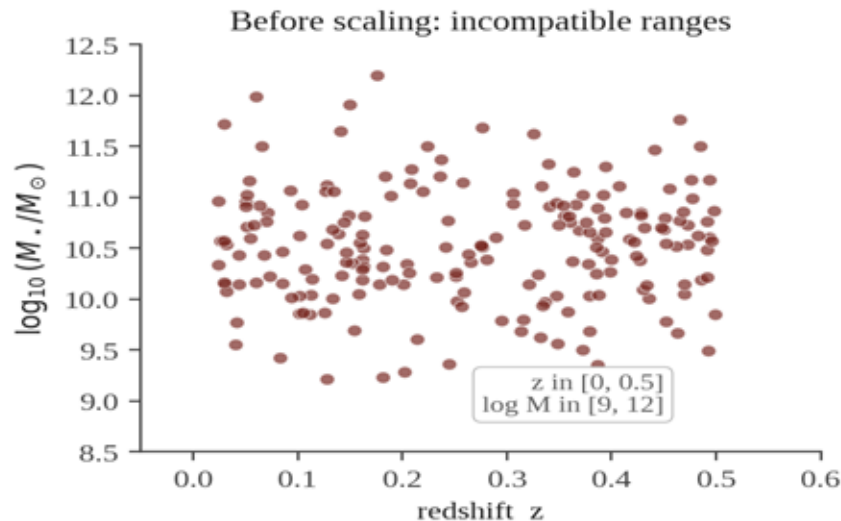
Step 2 — Apply the rule

$$|Z| = 3.6 > 3 \rightarrow \mathbf{\text{flag as outlier.}}$$

Why this trick works:

we used μ and σ from a known-clean sample. If we computed them *on the contaminated set including 19.8*, σ would inflate and the outlier could mask itself ($|Z| < 3$). $\rightarrow$ cross-check with IQR.

Why Feature Scaling Matters



The problem:

redshift z is in $[0, 0.5]$, stellar mass $\log M$ is in $[9, 12]$. Any distance-based or gradient-based model will let $\log M$ *dominate* the loss.

Models that care:

KNN, K-means, SVM, neural nets, PCA, gradient descent.

Models that don't: decision trees & random forests (next lecture).

Standardization vs Min-Max Scaling

Standardization (Z-score)

$$\mathbf{x}' = (\mathbf{x} - \mu) / \sigma$$

- Mean 0, variance 1.

Range is unbounded.

Use when: data is roughly Gaussian,
or you use PCA / linear models / NNs.

sklearn:

```
from sklearn.preprocessing import  
StandardScaler
```

Min-Max Scaling

$$\mathbf{x}' = (\mathbf{x} - \mathbf{x}_{\min}) / (\mathbf{x}_{\max} - \mathbf{x}_{\min})$$

- Output bounded in [0, 1].

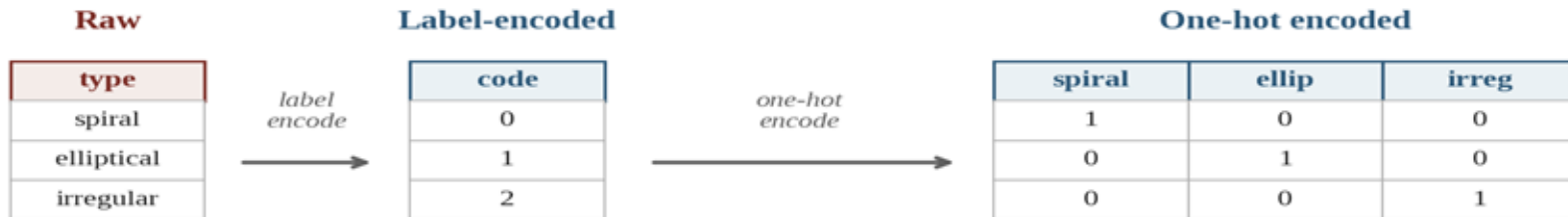
Preserves zero-relationships.

Use when: you need a fixed range,
e.g. neural-net image inputs, or PDFs.

sklearn:

```
from sklearn.preprocessing import  
MinMaxScaler
```

Encoding Categorical Features



Label Encoding

OK for **ordinal** data (E→So→Sa→Sb).

Wrong for **nominal** — the model will infer that $2 > 1 > 0$.

One-Hot Encoding

Default for **nominal** (spiral / ellip / irreg).

Cost: one new column per category — be careful for high-cardinality features.

Quick Check 2

QUIZ

Your dataset has a Hubble morphological stage column with values $E \rightarrow So \rightarrow Sa \rightarrow Sb \rightarrow Sc$. Which encoding preserves the most physical meaning?

A

One-hot encoding
— five binary columns

B

Label encoding (0,1,2,3,4)
— preserves the order

C

StandardScaler
— center and scale

D

Drop the column
— too tricky to encode

Pick A, B, C, or D — then advance.

Quick Check 2 — Answer

ANSWER

A

One-hot encoding

Loses the natural order — treats E and Sc as equally different from So.

C

StandardScaler

Scaling is for numeric data only — not categorical.

B

Label encoding (0..4)



Preserves the rank; use OrdinalEncoder in sklearn.

D

Drop the column

Throws away signal — Hubble stage carries real physics.

```
sklearn one-liner: from sklearn.preprocessing import OrdinalEncoder
OrdinalEncoder(categories=[["E", "S0", "Sa", "Sb", "Sc"]]).fit_transform(X)
```

Feature Selection — Pruning Redundancy

Not every feature helps. Some hurt (overfitting, collinearity, leakage).

1. Redundancy

If u-band and g-band are 95% correlated, you're paying twice for the same information.

2. Noise features

Features with no relationship to y just add variance — drop or test with a permutation check.

3. Leakage

A feature that secretly contains the target (e.g., a galaxy ID encoding redshift) must be dropped.

4. Curse of dimensionality

Fewer features → easier to fill feature space with limited data (Lec 1b).

Tools: correlation matrix, mutual information, `RandomForest.feature_importances_` (Lec 3b), recursive feature elimination.

PCA — A First Look

Principal Component Analysis

Find the directions in feature space along which the data *varies the most*.

Rotate the axes to align with those directions.

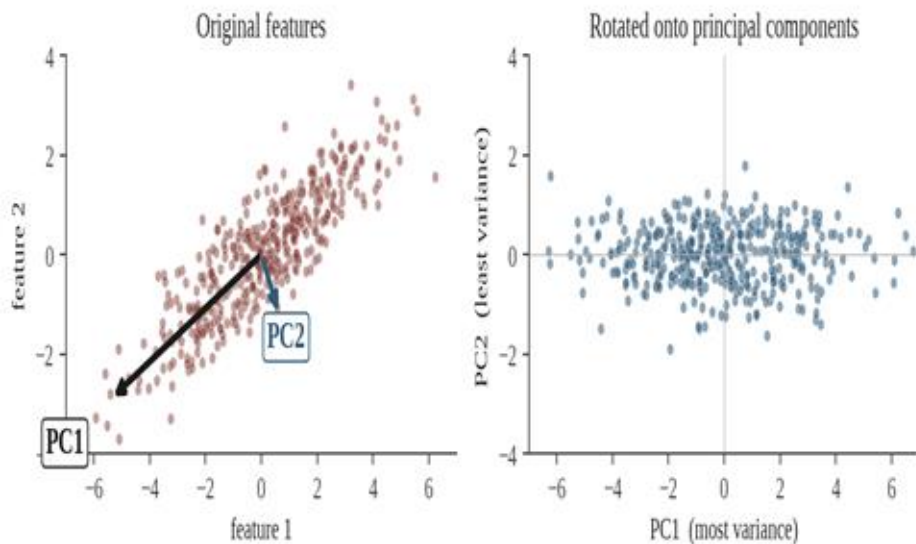
Keep only the top k components → dimensionality reduction.

Caveats:

linear only — won't see curved structure.

principal components are mixtures of original features, hard to interpret.

always standardize first!



Astro examples: compressing galaxy SEDs to a few components; finding eigenvectors of stellar spectra.

Code — A Full sklearn Pipeline

Chain every step into one object so it can't go out of sync.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, ["u", "g", "r", "i", "z"]),
    ("cat", OneHotEncoder(), ["galaxy_type"]),
])

X_train = preprocess.fit_transform(X_train)    # fit only on train!
X_test  = preprocess.transform(X_test)
```

Recap — The Data-Prep Checklist

1

Inspect for missingness

`df.isna().sum()` — drop, impute (mean/median/KNN), or flag.

2

Detect outliers

Z-score or IQR. Investigate before deleting; outliers might be the discovery.

3

Scale numeric features

StandardScaler (default) or MinMaxScaler. Fit on TRAIN only.

4

Encode categoricals

OrdinalEncoder for ordinal; OneHotEncoder for nominal.

5

Prune & (optionally) PCA

Drop redundant / leaky features. PCA for compact representations.

Next → **Lecture 3a:** train / validation / test splits and avoiding data leakage.